

HW12：可执行汇编

- 姓名：陈雨萱
- 学号：23300240034
- 专业：计算机科学与技术（拔尖人才培养）

关键技术实现

hw12实验的目的是在前序实验生成的伪汇编程序基础上，实现编译器后端的寄存器分配（Register Allocation）功能，完成从临时变量（Temp）到实际ARM寄存器的映射，最终生成可执行的ARMv7-A汇编代码。实验过程中需要完成汇编程序预处理、活跃变量数据流分析、冲突图（Interference Graph）构建、图着色寄存器分配以及溢出（Spill）变量处理等关键步骤，从而掌握现代编译器后端寄存器分配的基本原理与实现方法，理解数据流分析、图着色算法和寄存器资源管理在代码生成中的作用。

simcoafrespisel

该文件实现了**基于冲突图的图着色寄存器分配算法**，核心目标是将临时变量（Temp）映射到有限的 ARM 寄存器，或者决定其 Spill 到内存。通过 Simplify、Coalesce、Freeze、Spill、Select 五个阶段，先简化低度节点、合并可安全共享寄存器的 Move、冻结无法合并的节点，再在必要时选择溢出节点，最后逆序恢复并分配寄存器颜色。整个文件完成了编译器后端寄存器分配的核心逻辑，是实现高效可执行汇编代码的关键模块。具体函数如下：

1. 辅助函数有：
 - `** getAlias(int n, const map<int, set<int>>& coalescedMoves)` **函数用于查找节点在 Move 合并（Coalescing）后最终的代表节点，沿着 `coalescedMoves` 链条迭代直到找到顶层节点，保证在合并操作和最终着色时使用正确的主节点。
 - `** nodeDegree(int n, const map<int, set<int>>& graph)` **计算节点在冲突图中的度数，即与该节点相邻的冲突节点数量，为 Simplify、Coalesce、Spill 阶段判断节点是否可以安全处理提供依据。
 - `** adjacent(int u, int v, const map<int, set<int>>& graph)` **判断两个节点是否相邻，即它们是否在冲突图中存在边，辅助判断节点是否可以合并或是否会引入冲突。
 - `** okToCoalesce(int t, int r, const map<int, set<int>>& graph, int K)` **实现 George 判据，判断节点 `t` 是否可以与寄存器 `r` 合并。如果节点是机器寄存器、度数小于 `K` 或已经相邻，则允许合并，避免破坏图的可着色性。
 - `** conservative(const set<int>& nodes, const map<int, set<int>>& graph, int K)` **实现 Briggs 保守判据，统计高阶节点（度 $\geq K$ ）的数量，如果数量小于 `K`，则认为合并不会导致不可着色，从而允许合并普通临时变量。
 - `** adjacentNodes(int n, const map<int, set<int>>& graph)` **返回节点的所有邻居集合，便于在合并或判定过程中快速获取冲突节点集合。
 - `** combineNodes(Coloring* c, int u, int v)` **将节点 `v` 合并到节点 `u`，包括更新 `coalescedMoves`、合并邻居边、删除 `v` 节点以及移除与 `v` 相关的 Move 对，完成节点的真正合并操作。
 - `** freezeMovesFor(Coloring* c, int u)` **冻结节点 `u` 的所有 Move 操作，即删除与 `u` 相关的 Move 对，将节点转为普通变量，便于 Simplify 阶段继续处理。
2. Coloring 类核心函数则有：
 - `** bool Coloring::simplify()` **选择度数小于 `K` 的非机器寄存器节点，将其压入简化栈 `simplifiedNodes` 并从图中删除。该函数实现了图着色中的简化阶段，为逆序 Select 阶段做准备。
 - `** bool Coloring::coalesce()` **尝试合并 Move 相关节点，通过 George 或 Briggs 判据判断是否安全合并，如果可以，则调用 `combineNodes` 合并。该函数的目的是减少 Move 指令数量，同时保持图可着色。
 - `** bool Coloring::freeze()` **当 Move 合并无法继续时，冻结某些 Move 相关节点，将它们从 Move 集合中移除，使节点成为普通变量，从而让 Simplify 可以继续执行，打破僵局。
 - `** bool Coloring::spill()` **当 Simplify、Coalesce、Freeze 都无法处理图时，选择度数最大且非机器寄存器节点进行 Spill，将其压入简化栈并从图中删除，为后续着色阶段准备，同时标记高冲突变量以便生成访存操作。
 - `** bool Coloring::select()` **逆序处理简化栈 `simplifiedNodes` 中的节点，为每个节点分配寄存器颜色。如果没有可用颜色，则标记节点为 Spill。同时恢复被 Coalescing 合并的节点颜色，使最终得到 Temp $\rightarrow$ 寄存器或 Spill 的映射。

asmprog2colored

该文件实现了**汇编程序从临时变量到寄存器/Spill 的最终映射过程**。它以 Coloring 的寄存器分配结果为依据，将占位符临时变量替换成实际寄存器或栈偏移，并处理 Move 展开、溢出变量 Load/Store、栈操作修正以及标签格式化。整体上，文件完成了编译器后端最后一步，将 HW11 输出的伪汇编程序转化为可直接运行的彩色 ARMv7-A 汇编程序，是寄存器分配与 Spill 处理的核心实现模块。

1. 辅助函数实现了：
 - `** resolveAlias(int tempNum, const Coloring* coloring)` **用于查找一个临时变量在寄存器分配和 Move 合并（coalescing）后的最终代表节点，保证对已经合并或别名的 Temp 进行正确映射。
 - `** isSpilledTemp(int tempNum, const Coloring* coloring)` **判断一个临时变量是否被溢出到内存。通过 `resolveAlias` 找到主节点，再查看 Coloring 中的 spilled 集合。

- ```
** cleanFuncName(string funcName) **处理函数名格式，将特殊字符 ^ 替换为 $，去掉编译器内部生成的前缀和后缀，使输出汇编中的标签和函数名更规范、可读。
** replaceAll(string& text, const string& from, const string& to) **在字符串中全局替换子串 from 为 to，用于汇编指令中临时占位符的替换。
**

formatColoredAssem(const AssemInstr& instr, const string& funcName, const Coloring* coloring, const map<int, string>& spilledSrcRegs)
**将单条汇编指令中的临时变量占位符（d0, s0 等）替换为对应寄存器或 Spill 位置，同时处理标签、adr 指令格式。核心任务是生成最终可用的彩色（colored）汇编指令字符串。
** makeOper(const string& assem) **生成一条普通操作指令（Oper），用于生成中间或扩展指令，方便在 Spill 或 Move 展开时插入额外指令。
** patchStackInstruction(AssemInstr& instr, int extraBytes) **修正汇编指令中对栈指针的偏移量，将原有的栈操作（add/sub sp/fp）增加额外字节，用于存储 Spill 变量。
** expandInstruction(const AssemInstr& instr, const string& funcName, const Coloring* coloring, const map<int, int>& spillOffsets) **对原始汇编指令进行扩展，处理溢出变量（Load/Store）、Move 指令优化和占位符替换。返回一系列最终可执行的汇编指令。
```
2. 特别的，Coloring 相关工具函数是下面两个：
- ```
** getRegName(int colorNum) **根据寄存器编号返回寄存器名称字符串，例如 r0、r1。  
** getTempRegName(int tempNum, const Coloring* coloring) **根据临时变量和 Coloring 信息返回最终寄存器名。如果 Temp 被溢出，则返回 Spill 寄存器（如 r10），否则返回其分配的寄存器。
```
3. 核心函数是 `AsmProg* asmprog2colored(AsmProg* program, const vector<Coloring*>& colorings)`，它将整个汇编程序中的所有临时变量转换为最终寄存器或 Spill 内存位置。按函数遍历，每条指令调用 `expandInstruction` 和 `patchStackInstruction` 处理溢出变量和栈偏移，生成完整的彩色汇编程序 `AsmProg*`，输出可直接生成可运行 ARM 汇编代码。

predataflowpass

该文件实现的是**编译器后端数据流分析前的预处理步骤（pre-dataflow pass）**。其核心作用是在进入活跃变量分析和冲突图构建之前，修正汇编指令中缺失的隐式寄存器定义信息，尤其是函数调用指令对寄存器 r0-r3 的破坏性影响。通过显式补全这些寄存器的定义关系，可以保证后续 interference graph 构建的正确性，使寄存器分配阶段能够基于完整、准确的数据流信息进行分析与优化。

由于 ARM 调用约定规定函数调用会隐式修改前四个参数寄存器（r0-r3），但在初始汇编中这些寄存器可能没有被显式标注为定义（dst），因此 `preDataFlowPass(AsmProg* prog)` 函数会遍历所有函数的所有指令，在检测到调用指令后，将 r0-r3 中未出现在 dst 列表中的寄存器补充进去，从而保证后续数据流分析能够正确识别调用对寄存器的影响。

buildlg

这个代码文件的整体功能是实现编译器后端寄存器分配前的关键步骤——干涉图构建。它基于数据流分析得到的活跃变量信息，推导出变量之间的冲突关系，并将这些关系表示为图结构（Interference Graph），用于后续的图着色寄存器分配算法。同时，它还额外记录 move 指令相关的变量对，为后续的寄存器合并（coalescing）优化提供依据。整体流程可以概括为：先做活跃变量分析，再基于 liveness 构建冲突图，最后输出每个函数对应的干涉图集合。具体函数如下：

- `addIgEdge` 函数的作用是在干涉图中添加一条无向边，用于表示两个临时变量之间存在冲突关系。实现上使用 `map<int, set<int>>` 作为邻接表结构，如果两个变量编号相同则直接忽略，避免自环。否则就在 `u` 和 `v` 之间互相加入对方，构成双向关系。其本质含义是：这两个变量在寄存器分配中不能被分配到同一个寄存器，因为它们在某些程序点上可能同时处于活跃状态。
- `buildIg` 函数用于为单个函数构建干涉图，是整个文件的核心逻辑。首先它收集函数中所有出现过的临时变量，确保图中节点完整。然后从后向前遍历指令，利用已计算好的活跃变量信息（liveout）来构建冲突关系：如果某个变量在当前点被定义，而另一个变量仍然活跃，那么这两个变量之间就会加入干涉边。同时，对于 move 指令（如 `x = y`），函数会记录 move pair，并在 live 集合中暂时移除源变量，以便后续进行寄存器合并优化。最后通过 def/use 关系不断更新 live 集合，实现标准的反向活跃变量传播。
- `buildIgProg` 函数是整个程序级别的入口，用于为一个程序中的所有函数构建干涉图集合。它遍历 `AsmProg` 中的每个函数，对每个函数先进行活跃变量分析（调用 `computeLiveness`），然后调用 `buildIg` 构建对应的干涉图，并将结果保存到 `vector` 中返回。如果程序为空或没有函数，则直接返回空结果。

开发过程和测试结果

开发过程如下：



make run 之后成功编译，得到的终端输出是：

```
Node 9: 11 14
Node 10: 11 14
Node 11: 0 1 2 3 4 5 6 7 8 9 10 13 14
Node 13: 11 14
Node 14: 4 5 6 7 8 9 10 11 13
Simplified nodes (reversed order from the simplification process):
Coalesced moves:
Remaining move pairs:
Spilled temps: 10000 10201
Colored nodes:
0 -> 0
1 -> 1
2 -> 2
3 -> 3
4 -> 4
5 -> 5
6 -> 6
7 -> 7
8 -> 8
9 -> 9
10 -> 10
11 -> 11
13 -> 13
14 -> 14
115 -> 0
116 -> 0
117 -> 0
118 -> 0
119 -> 0
10100 -> 1
10101 -> 4
10102 -> 4
10103 -> 0
10104 -> 1
10200 -> 0
10202 -> 0
11000 -> 0
```

```
===== APPLYING COLORS FOR k=9 =====
Writing colored assembly to: k9/optloopivtest6.colored.s
```

```
Done.
```

```
root@cyx:~/fducompilerh2026/fducompilerh2026/HW12#
```

对于所有测试文件，编写了一个自动测试脚本来测试各种情况下，k2/5/9得到的输出是否与测试函数理论上的输出一致，以及是否稳健。得到的脚本输出为全部通过。：

```
$ cd /root/fducompilerh2026/fducompilerh2026/HW12/test && chmod +x
comprehensive_check.sh && ./comprehensive_check.sh 2>&1 | tee
/tmp/comprehensive_result3.txt | tail -40
OK optloopivtest3(0,1) k=5
OK optloopivtest3(1,1) k=5
OK optloopivtest3(2,3) k=5
OK optloopivtest3(1,10) k=5
OK optloopivtest3(5,5) k=5
OK optloopivtest3(10,1) k=5
OK optloopivtest3(100,50) k=5
OK optloopivtest1(0) k=9
OK optloopivtest2(0) k=9
OK optloopivtest3(0,1) k=9
OK optloopivtest3(1,1) k=9
OK optloopivtest3(2,3) k=9
OK optloopivtest3(1,10) k=9
OK optloopivtest3(5,5) k=9
OK optloopivtest3(10,1) k=9
OK optloopivtest3(100,50) k=9
```

```
===== 7. 死循环检测: optloopivtest3(10,0) =====
OK optloopivtest3(10,0) k=2 (timeout as expected)
OK optloopivtest3(10,0) k=5 (timeout as expected)
OK optloopivtest3(10,0) k=9 (timeout as expected)

===== 8. k=2 压力测试: insttest1~3 连跑 10 次 =====
OK insttest1 k=2 stress x10
OK insttest2 k=2 stress x10
OK insttest3 k=2 stress x10

===== 9. 边界 k 一致性 =====
OK CONSIST fibonacci(0)
OK CONSIST fibonacci(-1)
OK CONSIST fibonacci(10)
OK CONSIST optloopivtest3(0,1)
OK CONSIST optloopivtest3(10,1)
OK CONSIST optloopivtest3(100,50)
OK CONSIST optloopivtest1(0)
OK CONSIST optloopivtest2(0)

===== SUMMARY =====
PASS=194 FAIL=0 CRASH=0 TIMEOUT=0 COMPILE_FAIL=0 TOTAL=194
VERDICT: ALL_PASSED
```

参考资料

- 大模型：DeepSeek/Cursor/ChatGPT，协助理解虎书内容和修改代码中遇到的汇编语序和合并的合法性问题，以及k=2时或者其他情况下代码不稳健问题。